INSTITUTO INFNET

ESCOLA SUPERIOR DE TECNOLOGIA

GRADUAÇÃO EM ENGENHARIA DE SOFTWARE



Fundamentos de Desenvolvimento com C#

TP1

Aluno: Ruan Carllo Silva

2 de nov. de 2026

Exercício 1

Comando utilizado:

```
dotnet run --project Ruan_Silva_DR2_TP1.Exercicio1.AmbienteEstacionamento
```

Saída obtida:

```
Nome do sistema atual: Unix 26.0.1
Versão .NET: 10.0.14
Diretório de trabalho atual: /Users/ruan/Desktop/Ruan_Silva_DR2_TP1
```

Exercício 2

No universo do C# com .NET, o mesmo código pode ser executado em sistemas operacionais diferentes (Windows, Mac, Linux), devido a uma série de fatores. Primeiramente, o compilador da linguagem gera a IL (“Intermediate Language”), um conjunto de instruções independente de processador e de sistema operacional, que forma um assembly (geralmente um arquivo .dll). Em seguida, o CLR (“Common Language Runtime”) específico de cada sistema operacional carrega esse assembly e, em tempo de execução, com o seu compilador JIT (“Just-In-Time”), traduz a IL para o código de máquina específico daquele processador e sistema operacional em uso.

Exercício 3

1) Ao realizar o build do projeto, com o comando a seguir, foram gerados os seguintes arquivos na sua pasta bin/Debug/net10.0:

Comando utilizado:

```
dotnet build Ruan_Silva_DR2_TP1.Exercicio1.AmbienteEstacionamento
```

Arquivos gerados:

- Ruan_Silva_DR2_TP1.Exercicio1.AmbienteEstacionamento
- Ruan_Silva_DR2_TP1.Exercicio1.AmbienteEstacionamento.dll
- Ruan_Silva_DR2_TP1.Exercicio1.AmbienteEstacionamento.deps.json
- Ruan_Silva_DR2_TP1.Exercicio1.AmbienteEstacionamento.runtimeconfig.json
- Ruan_Silva_DR2_TP1.Exercicio1.AmbienteEstacionamento.pdb

Cada um desses arquivos exerce um papel específico no ecossistema .NET: o .dll, é o assembly propriamente dito, contendo o código IL gerado pelo compilador e os metadados do programa. Já o .pdb (“Program Database”) armazena informações de depuração, permitindo mapear o código IL de volta ao código-fonte original. Por sua vez, o .deps.json descreve as dependências do projeto, ou seja, quais bibliotecas e versões são necessárias para a execução. Além deles, o .runtimeconfig.json contém as configurações do runtime, como a versão do .NET exigida e opções de execução. Por

fim, mas não menos importante, o arquivo que, no meu MacOS e também no Linux fica sem uma extensão, mas no Windows normalmente aparece como .exe, é um executável nativo gerado pelo SDK que serve de ponto de entrada, localizando o runtime configurado e carregando o assembly gerenciado no CLR.

2) Em resumo, nesse processo, o SDK ("Software Development Kit") do .NET fornece todo o conjunto de ferramentas necessárias para desenvolver, compilar e executar a aplicação, incluindo o próprio compilador e a CLI (dotnet). O compilador (Roslyn) é o responsável por traduzir o código C# escrito no arquivo Program.cs para a IL, gerando o assembly (.dll). O CLR ("Common Language Runtime") é o ambiente de execução que carrega esse assembly e gerencia a execução do programa, incluindo a compilação JIT ("Just-In-Time"). Por fim, a BCL ("Base Class Library") é a biblioteca de classes base do .NET, que fornece tipos e funcionalidades fundamentais (como Console, String, listas, arquivos, etc.) utilizados tanto pelo código quanto pelo próprio runtime.

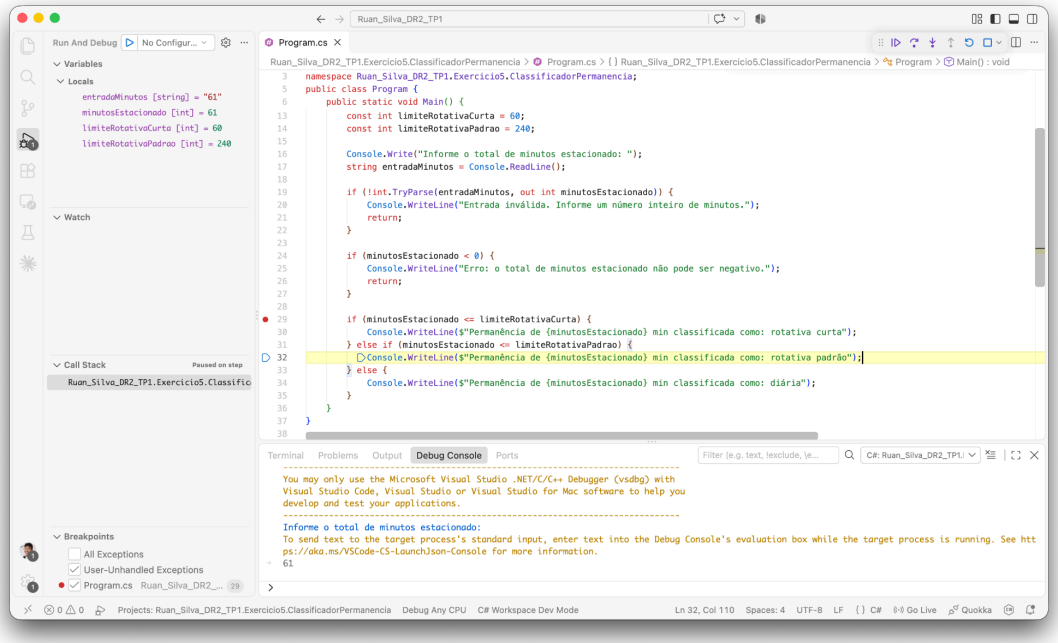
3) Portanto, em relação ao ciclo de vida completo do programa, ele ocorre da seguinte forma: primeiramente, escrevemos o código-fonte centralizado no arquivo Program.cs. Em seguida, ao executar "dotnet build", o compilador Roslyn traduz esse código para IL, gerando o assembly (.dll) e os arquivos auxiliares (.pdb, .deps.json, .runtimeconfig.json). Depois, ao executar o programa, o CLR carrega o assembly em memória, e, em tempo de execução, o compilador JIT do CLR traduz a IL para código de máquina específico do processador e sistema operacional em uso. Com isso, o código de máquina é executado, dando vida ao programa.

Exercício 4

A calculadora de tarifa usou divisão inteira e o operador "%" para separar horas cheias e minutos restantes, e o arredondamento para cima foi feito seguindo o raciocínio: quando há minutos restantes, soma-se uma hora às horas cheias, o que cobra qualquer fração iniciada como hora cheia e mantém o caso de zero minutos sem cobrança. As entradas foram lidas com "TryParse" e validadas nos caminhos negativos, com mensagem de erro e encerramento antes de qualquer cálculo, e a comparação com o teto diário usou os operadores relacionais ">", "<" e o "else" para o caso de igualdade.

Exercício 5

O classificador de permanência usou "if/else" com limites definidos no código e justificados nos comentários. As fronteiras foram fechadas no limite inferior ("<= 60" e "<= 240"), de modo que exatamente 60 minutos cai na faixa curta e exatamente 240 minutos na padrão. A entrada foi lida com "TryParse" e tratada em dois caminhos negativos distintos, um para texto inválido e outro para valor negativo, cada um com mensagem própria, o que melhora o diagnóstico. O valor da variável de permanência foi inspecionado no debugger do Visual Studio Code em tempo de execução para confirmar que o "TryParse" gravou o valor esperado antes da comparação.



Exercício 6

O classificador foi envolvido em um laço "while (true)" com encerramento por valor sentinela "-1". A ordem das validações foi decisiva: a comparação com a sentinela vem antes da rejeição de negativos, caso contrário o "-1" cairia no ramo de erro e o laço nunca terminaria. O valor sentinela não é contabilizado porque o "break" ocorre antes de qualquer classificação, e cada contador só é incrementado dentro do ramo correspondente. Entradas inválidas e negativos que não sejam a sentinela usam "continue", voltando ao topo do laço sem afetar os contadores, e o total de veículos é derivado ao final pela soma dos três contadores, sem um quarto contador redundante.

Exercício 7

O menu foi implementado com “do-while”, garantindo que ele apareça ao menos uma vez e volte a ser exibido após cada operação até a escolha de sair, e a opção lida foi validada com “TryParse” antes de entrar no “switch”, que possui “default” explícito para opções fora da faixa. A lógica dos exercícios anteriores foi extraída para métodos reutilizáveis, como o cálculo de tarifa, a conversão de duração e a classificação de permanência, sem duplicação de código. As leituras de minutos e de valor por hora foram centralizadas em métodos que retornam “null” em qualquer falha de validação, e os métodos de execução só seguem adiante quando o retorno não é nulo, o que evita validação espalhada pelo código.

Exercício 8

O programa consolidado reuniu todas as operações em um único executável, com a opção "Sobre" usando “Environment.Version” para exibir a versão do .NET em uso e uma linha explicando que a portabilidade se deve à IL, que é independente de plataforma, e do CLR de cada sistema operacional, que traduz essa IL para código de máquina por meio do JIT.

Uso de IA

Para fazer este trabalho, usei o DeepSeek chat para me ajudar a estruturar o projeto da forma como ficou no ZIP, aprendendo como se usa a flag ”--project” do “dotnet run” para executar cada programa do diretório raíz deste TP. Além disso, foi usada para relembrar o significado de várias siglas, como JIT e CLR.